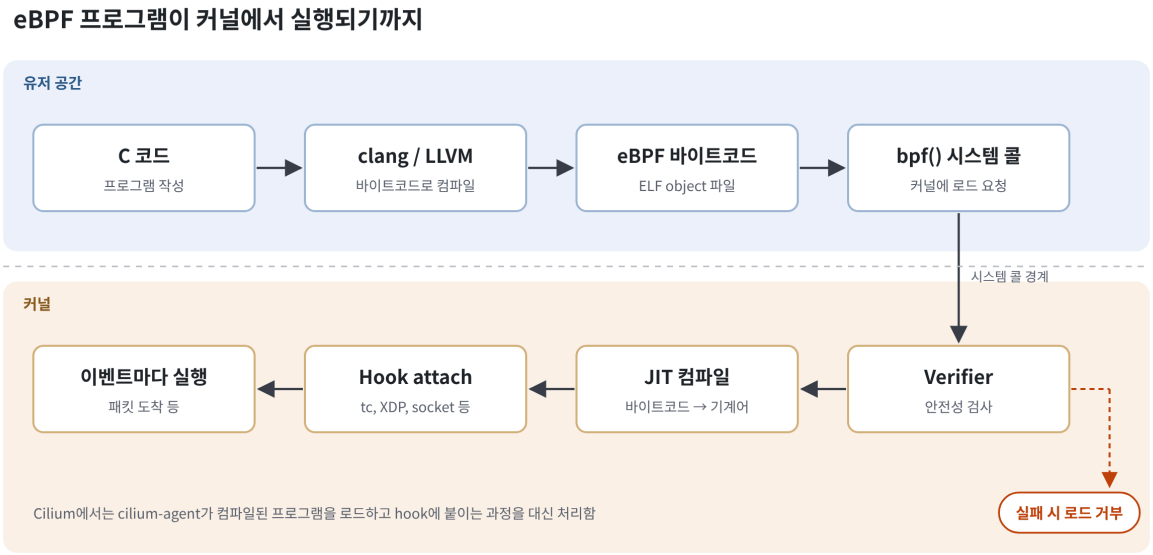
Week 7. eBPF Deep Dive & Hubble

eBPF 동작 원리

동작 과정



- C로 작성한 코드를 clang/LLVM으로 eBPF 바이트코드로 컴파일함
- bpf() 시스템 콜로 커널에 넘김
- 커널이 verifier로 안전한지 검사하고, 통과하면 JIT으로 기계어로 바꿈
- 지정한 hook에 붙이고, 이후 그 hook을 이벤트가 지날 때마다 실행됨

Cilium을 쓰면 이 과정을 직접 할 일이 없음!! cilium-agent가 프로그램과 map을 커널에 올리고 hook에 붙임

Verifier

- eBPF가 커널 모듈과 다른 이유는 결국 verifier 때문
  - 커널 모듈은 잘못 짜면 커널이 같이 죽음
  - eBPF는 로드 단계에서 걸러짐

- 검사하는 것
  - 로드하는 프로세스에 권한(root 또는 CAP_BPF)이 있는지
  - 프로그램이 반드시 끝나는지 (무한 루프 X, 종료 조건이 보장된 bounded loop만 O)
  - 초기화 안 된 변수를 쓰거나 메모리 범위 밖에 접근하지 않는지
  - 프로그램 크기와 복잡도가 한도 안인지
- 헷갈리기 쉬운 점
  - verifier는 커널을 망가뜨리지 않는지만 봄
  - 프로그램이 무슨 일을 하는지까지 판단하는 보안 도구는 아님

JIT

- 검증이 끝난 바이트코드를 CPU 기계어로 바꿈
- 그래서 커널에 원래 있는 코드와 비슷한 속도로 동작
- `/proc/sys/net/core/bpf_jit_enable` 값으로 켜져 있는지 확인 가능 (1이면 활성화)

Hook

- eBPF는 이벤트 기반이라 어디에 붙느냐 = 언제 실행되느냐

Cilium이 쓰는 hook

Hook	실행 시점	Cilium에서 쓰는 곳
XDP	NIC 드라이버에서 패킷을 받자마자	prefilter, 빠른 drop
tc ingress/egress	스택 초기 처리 후, L3 이전	L3/L4 정책, endpoint로 redirect
Socket operations	TCP 상태가 바뀔 때	ESTABLISHED 전환 감지
Socket send/recv	TCP 소켓이 send할 때마다	같은 노드 소켓끼리 빠른 redirect

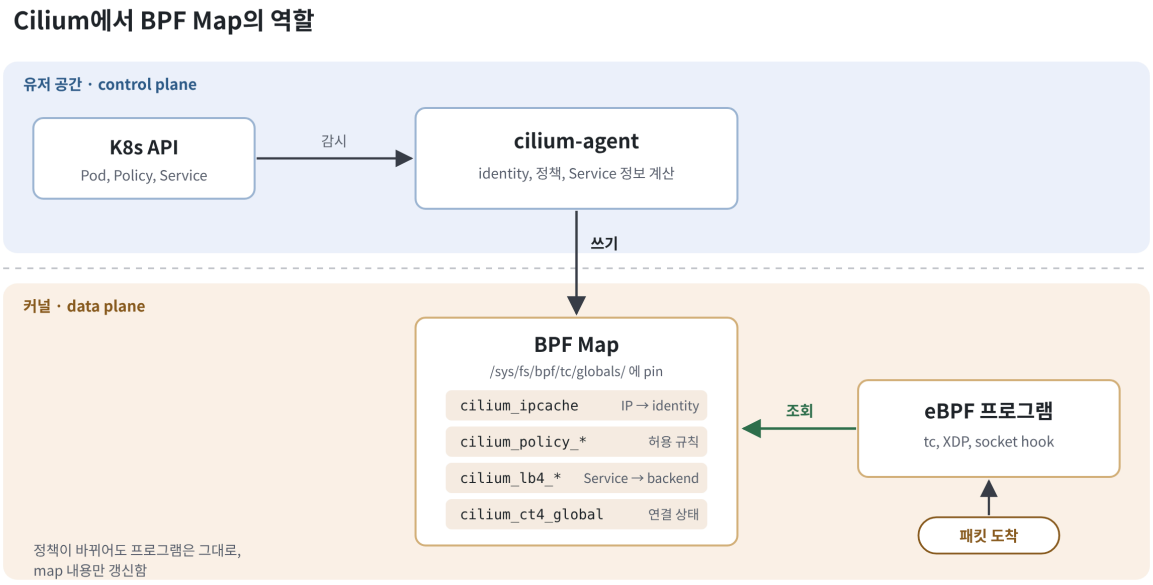
- Pod 트래픽은 대부분 tc에서 처리됨
  - Pod veth의 호스트 쪽(lxc 인터페이스) tc ingress에 프로그램을 붙임
  - Pod에서 나가는 트래픽을 전부 여기서 보고 정책을 걸 수 있음
- eBPF 프로그램은 아무 커널 함수나 부를 수 없음
  - 커널이 제공하는 helper 함수만 사용 가능
  - map 조회, 패킷 수정도 helper를 통해서 함

BPF Map & Cilium Datapath

BPF Map

- eBPF 프로그램은 패킷 하나 처리하고 끝나서 상태를 기억하지 못함
- 연결 정보, 정책처럼 계속 유지돼야 하는 데이터는 BPF Map에 둠
- 특징
  - eBPF 프로그램끼리 공유 가능
  - 유저 공간 프로그램도 시스템 콜로 읽고 쓸 수 있음
  - 종류: Hash, Array, LRU, LPM, Ring Buffer 등

Cilium에서 Map의 역할



- map이 control plane과 data plane을 이어줌
  - cilium-agent: K8s API 감시 → identity, 정책, Service 정보 계산 → map에 씀
  - eBPF 프로그램: 패킷마다 map을 읽기만 함
- 정책이 바뀌어도 프로그램은 그대로, map 내용만 바꾸면 됨
- iptables와 비교
  - iptables: 규칙을 위에서부터 차례로 훑음
  - map: key로 바로 찾음 → 규칙이 많아져도 조회 비용이 크게 늘지 않음

- map 위치: `/sys/fs/bpf/tc/globals/` 아래에 pin됨
  - pin해 두면 만든 프로세스가 내려가도 map이 커널에 남음
  - bpftool, cilium-dbg 같은 도구가 경로로 접근 가능

Cilium이 쓰는 주요 Map

Map	이름	기본 한도	들어 있는 것
IP cache	`cilium_ipcache`	512k	IP/CIDR와 identity 매핑
Policy	`cilium_policy_*` (endpoint별)	16k	허용하는 identity + port + protocol
Service LB	`cilium_lb4_services_v2`	64k	Service frontend와 backend 슬롯
Service Backends	`cilium_lb4_backends_*`	64k	실제 backend Pod IP:port
Connection Tracking	`cilium_ct4_global` 등	TCP 512k / UDP 256k	연결 상태

- 이름 뒤 버전 접미사는 Cilium 버전마다 다를 수 있음 → `/sys/fs/bpf/tc/globals/`에서 직접 확인

ipcache

- 정책 판단의 시작점
- 패킷에는 IP만 있고 identity는 없음 → ipcache에서 IP로 identity를 찾음
- Pod가 새로 뜨거나 IP가 바뀌면 ipcache만 고치면 됨 (정책 map은 그대로)
- Week 5에서 본 IP가 아닌 identity 기준 정책을 실제로 연결해주는 게 ipcache

Policy map

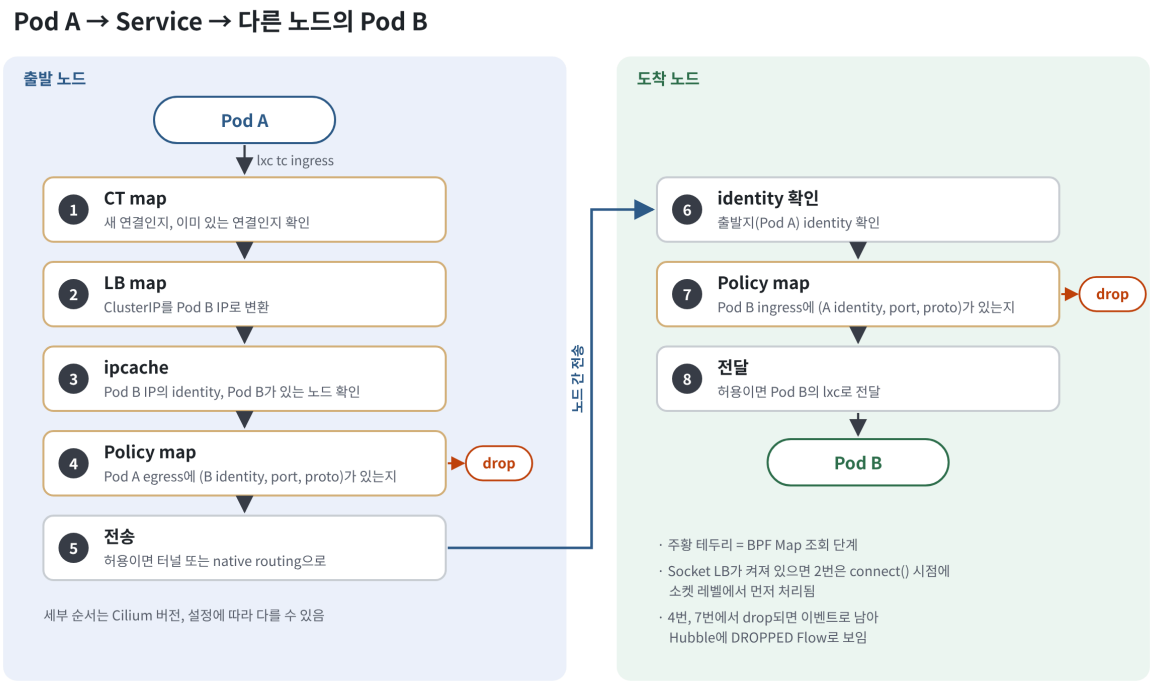
- endpoint마다 따로 있음
- (상대 identity, 포트, 프로토콜, 방향) 조합으로 항목이 있으면 허용, 없으면 drop
- endpoint당 한도 16k
  - identity를 많이 고르는 정책(CIDR를 /32로 잘게 쪼갠 규칙 등)은 map이 꽉 찰 수 있음

LB map

- kube-proxy의 Service 규칙 역할
- Service 하나당 항목 수 ≈ backend 수 × 포트 수

Map을 참조해서 패킷을 처리하는 과정

Pod A가 Service를 거쳐 다른 노드의 Pod B로 요청을 보내는 경우 (세부 순서는 버전, 설정에 따라 다름)



- Socket LB(kube-proxy replacement)가 켜져 있으면
  - 2번은 패킷 단계까지 안 오고 `connect()` 시점에 소켓 레벨에서 먼저 처리됨
- 4번이나 7번에서 drop되면 이벤트로 남음 → Hubble에서 DROPPED Flow로 보임

Hubble & Troubleshooting

eBPF Event에서 Hubble Flow까지

```
eBPF 프로그램 (trace / drop / policy verdict 이벤트)
  → perf ring buffer
  → cilium-agent monitor
  → Hubble server (agent 내장, 이벤트를 Flow로 변환)
  → Hubble ring buffer
  → Observer gRPC
```

```
→ Hubble Relay (모든 노드의 Hubble에 4244 포트로 연결)
→ hubble CLI / UI
```

- Hubble이 패킷을 따로 캡처하는 줄 알았는데 아니었음
  - datapath가 이미 남기고 있던 이벤트를 받아 Flow로 바꿔 보여주는 구조
- Flow에 Pod 이름, namespace, label이 붙는 이유
  - datapath 이벤트에는 IP와 identity 정도만 있음
  - Hubble server가 agent의 endpoint, identity 정보로 채워 넣음
- 이 구조 때문에 생기는 특징
  - Cilium이 관리하지 않는 Pod(hostNetwork Pod, Cilium보다 먼저 뜬 Pod)는 Hubble에도 안 보임
  - ring buffer가 차면 오래된 Flow부터 덮어씀 → 예전 Flow가 안 보여도 정상
  - `cilium-dbg monitor`도 같은 이벤트를 봄 → Hubble이 안 될 때 대신 사용 가능

Flow 읽기

```
default/tiefighter:50214 -> default/deathstar-...:80 to-overlay FORWARDED (TCP Flags:
SYN)
```

- 순서: 출발지 → 목적지 → 관측 지점 → verdict → L4 정보
- 관측 지점: 이벤트가 어디서 찍혔는지
  - `to-endpoint`: 컨테이너로
  - `to-stack`: 호스트 스택으로
  - `to-overlay`: 터널로
  - `to-proxy`: L7 proxy로
- verdict
  - 주로 보는 건 FORWARDED, DROPPED
  - 그 외 REDIRECTED(L7 proxy로 넘김), AUDIT(audit 모드라 통과했지만 원래라면 막혔을 것) 등
- `o json`으로 보면 더 자세히 나옴. 주로 보는 필드
  - `drop_reason_desc`

- `traffic_direction` (INGRESS / EGRESS)
- `source.identity`, `destination.identity`

FORWARDED / DROPPED

FORWARDED

- 정상 통신이면 TCP handshake가 FORWARDED로 쭉 이어짐
- 다른 노드로 가는 경우 경로도 같이 보임
  - 출발 노드: `to-overlay`
  - 도착 노드: `to-endpoint`

DROPPED

- drop reason부터 확인해야 함. 이유에 따라 원인이 완전히 다름
  - Policy denied: NetworkPolicy에 걸림 (가장 흔함)
  - CT: Map insertion failed: CT map이 가득 참
  - Service backend not found: Service에 backend가 없음
  - Stale or unroutable IP: ipcache와 실제 상태가 안 맞음
- L4에서 막혔는지 L7에서 막혔는지 Flow 모양으로 구분 가능
  - L4 정책: SYN이 DROPPED로 여러 번 찍힘 (클라이언트가 재전송하기 때문)
  - L7(HTTP) 정책: TCP 연결은 FORWARDED, `http-request DROPPED`로 나옴

Hubble로 통신 문제 추적하기

1. Pod가 Cilium 관리 대상인지 확인
   - `cilium status`의 `Cluster Pods` 항목
   - `cilium-dbg endpoint list`
2. 해당 Pod의 Flow 확인
   - `hubble observe --pod <ns>/<pod>`
3. 막힌 것만 걸러서 drop reason 확인
   - `hubble observe --pod <ns>/<pod> --verdict DROPPED`
4. Policy denied면 어느 쪽에서 막혔는지 구분

- `traffic_direction`과 양쪽 identity 확인
- 출발지 egress인지, 목적지 ingress인지

5. 정책이 실제로 어떻게 들어갔는지 확인
   - `cilium-dbg endpoint get <id>`의 `status.policy.realized`
   - `cilium-dbg bpf policy get <id>`
     DROPPED가 없는데도 통신이 안 되는 경우

- Flow 자체가 없음 → Cilium이 못 보는 경로
- FORWARDED인데 응답이 없음 → 애플리케이션 문제

질문

▼ 1. XDP가 tc보다 앞단이라 더 빠르다는데, Pod 정책은 왜 XDP가 아니라 tc에서 걸까?

- XDP는 NIC 드라이버에서 패킷을 받는 시점에 실행됨
  - 네트워크 스택 처리 전이라 빠르지만, 패킷 데이터 말고는 아는 게 거의 없음
  - 그래서 Cilium은 XDP를 prefilter, 빠른 drop(DDoS 방어) 용도로 씀
- tc는 스택이 초기 처리를 한 뒤, L3 전에 실행됨
  - 패킷에 딸린 메타데이터 대부분에 접근할 수 있음
  - Cilium 문서도 L3/L4 endpoint 정책 적용, endpoint로 redirect 같은 노드 로컬 처리에 tc가 적합하다고 설명함
- Pod 쪽 구조와도 맞음
  - Pod는 veth로 호스트와 연결됨
  - 호스트 쪽 veth(lxc)의 tc ingress에 붙이면 Pod에서 나가는 트래픽을 전부 볼 수 있음
- 정리하면 XDP는 빠르게 걸러내는 용도, tc는 Pod 단위 정책을 거는 용도로 역할이 나뉨

▼ 2. BPF Map에 한도가 있다고 했는데, 꽉 차면 어떻게 될까?

- 공식 문서 기준으로 한도를 넘는 insert는 실패함
- 그 결과는 map 종류마다 다름
  - CT map: 새 연결 항목을 못 만들어서 `CT: Map insertion failed`로 drop됨

        - 문서는 GC 주기(`-conntrack-gc-interval`)를 줄이거나 CT map 크기를 늘리라고 안내함
    - LB map: Service 변경을 반영하지 못해서 Service IP 통신이나 새 Service 생성에 문제가 생길 수 있음
    - Policy map: endpoint당 16k라서 identity를 많이 고르는 정책이 있으면 꽉 찰 수 있음
        - `cilium_bpf_map_pressure` 메트릭으로 미리 감지함
- 크기를 키울 수는 있지만, 설치 후 LB map 크기를 바꾸고 재시작하면 map을 다시 채우는 동안 연결이 끊길 수 있음
- 그래서 설치 전에 크기를 미리 정하라고 권장

▼ 참고 자료

- What is eBPF? (ebpf.io)
- Cilium – eBPF Datapath Introduction
- Cilium – Life of a Packet
- Cilium – eBPF Maps
- Cilium – Hubble internals
- Cilium – Inspecting Network Flows with the CLI
- Cilium – Troubleshooting